

Pirate – Grouper

Matteo

Abstract

The following document provides an overview of the addition of the “Grouper” module to `pirate`. By “Grouper” I mean the module that extracts significant excesses from the S/N maps produced by `pirate` and identifies detections of the same event. Grouping is performed in two stages, namely “Peakfinding” and “Clustering”. The Peakfinder selects significant local maxima within each map, whereas Clustering associates the surviving points reported by different search trees. I describe the choices adopted for these two stages, how the code handles bursts extending across consecutive chunks, and the Python implementation used for offline and online processing. Finally, I present a preliminary assessment of the computational cost of the Grouper.

Input data

The input to the Grouper consists of the arrays `out_max` and `out_argmax` produced by `pirate`. For each search tree, `out_max` contains the maximum S/N found within each coarse DM–time cell. The matching `out_argmax` array stores the trial responsible for that maximum, including the temporal profile, frequency multiplet, fine time and additional DM index. These quantities are used to recover the candidate DM, arrival time, width and frequency interval after selecting a peak.

The maps have dimensions (beam, DM, time). Each tree has its own DM and time resolution, which is preserved throughout Grouping. The native spacings of tree j are

$$\Delta\text{DM}_j = \frac{\text{DM}_{\text{max},j} - \text{DM}_{\text{min},j}}{N_{\text{DM},j}}, \quad \Delta t_j = \frac{N_{\text{in}}}{N_{t,j}} t_{\text{samp}}.$$

Here N_{in} is the number of input samples per chunk and $N_{t,j}$ is the number of coarse time bins in the output map. The arrival time obtained from the `argmax` decoder is referred to the lowest frequency of the complete observing band. It includes the finer trial information and should therefore be distinguished from the time coordinate of the coarse map cell.

In offline processing, the S/N and `argmax` maps can be saved as ASDF files, together with the dedispersion configuration, plan and timing information. The Grouper can therefore be rerun without repeating the dedispersion. Online, the same arrays are read directly from the GPU output buffers of `pirate`.

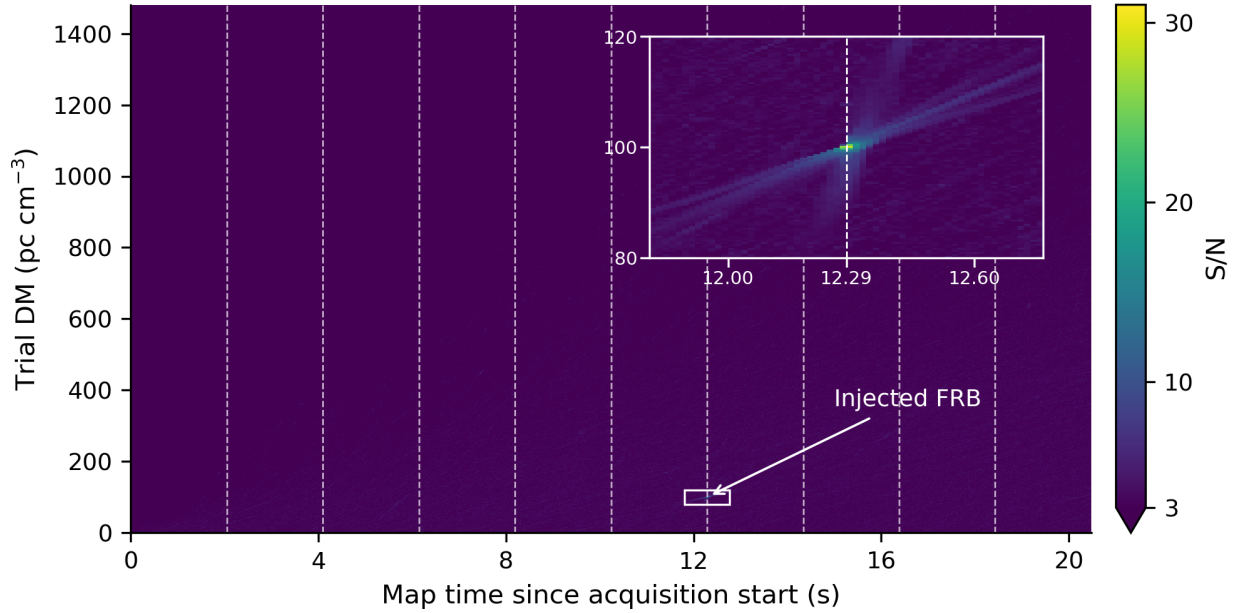


Figure 1. Stacked Tree 0 S/N maps from a simulated CHORD acquisition of ten chunks. Each chunk contains 2048 input samples at 1 ms, for a total of 20.480 s. I injected a burst with $DM = 100 \text{ pc cm}^{-3}$, temporal standard deviation 2 ms and target $S/N = 30$ into the CHORD configuration of 28,160 channels covering 300–1500 MHz. Its arrival time at 300 MHz is 12.288 s, at a chunk boundary. The complete dispersed pulse lies inside the acquisition. Tree 0 has 4096 DM rows and 128 time columns per chunk, corresponding to $0.36155 \text{ pc cm}^{-3}$ and 16 ms per map cell. Dashed lines separate the chunks; the inset enlarges the burst using the same time and DM units. The plotted time is the coarse map coordinate; the colour scale saturates below $S/N = 3$.

Algorithm choices

A burst does not produce a single pixel above threshold in a DM–time map. At nearby trial DMs the signal is still detected, although with a lower S/N and a shifted arrival time. Several neighbouring time cells can also exceed the threshold, especially for a bright or broad burst. The resulting region contains many detections of the same signal. If all these pixels were passed directly to Clustering, most of the work would be spent comparing redundant points.

The number of threshold crossings also depends on the burst brightness and on the amount of RFI in the maps. Consequently, grouping every above-threshold pixel would make the candidate workload depend strongly on the size of these regions. I first reduce each map to a smaller set of representative peaks. This is the purpose of the Peakfinder: it asks whether a pixel is significant and whether a stronger response is present in the neighbourhood expected for a dispersed pulse.

This first selection is performed independently for each tree. However, a burst may be detected in more than one search tree. A pulse may be reported by the full-band and early-trigger trees belonging to the same primary-tree family, with slightly different DM, time and width estimates. Peakfinding within one map cannot recognise that these detections refer to the same event. The surviving points are decoded and then associated in physical DM and time coordinates.

The two stages address different sources of duplication. Peakfinding reduces the extended response within each native map; Clustering combines eligible detections across trees. This separation allows me to use the full map for the neighbourhood comparison and then work with compact candidate arrays.

Peakfinder

To define the peakfinding neighbourhood, consider the shape produced by a burst when the trial DM differs from its true value. The channels are no longer aligned exactly, so the response broadens in time. The range of possible time offsets grows with the DM offset and reverses direction when the sign of that offset changes. In the DM–time plane this produces two opposing wings, which I refer to as a bow tie.

I use this shape to construct a Boolean footprint around each candidate pixel (see Fig. 2). A true entry in the footprint identifies a neighbouring pixel that participates in the local-maximum comparison. For a DM difference δDM , the residual delay at frequency f is

$$\delta t(f) = K_{\text{DM}} \delta\text{DM} (f_{\text{ref}}^{-2} - f^{-2}),$$

where $K_{\text{DM}} = 4148.808 \text{ s MHz}^2 (\text{pc cm}^{-3})^{-1}$ and frequencies are expressed in MHz. I set the reference frequency to the lower edge of the complete band. Evaluating this relation at the two band edges gives the time interval spanned by the bow tie at a given DM offset. I then divide by the native time spacing of the tree to express the interval in map bins.

The parameter `dm_reach` sets how many native DM rows are considered on either side of the centre. A value of 8 therefore gives a footprint with 17 rows. The parameter `waist_bins` adds a small time allowance around the central ridge. At zero DM offset, the footprint spans from `-waist_bins` to `+waist_bins`; away from zero, this allowance extends the outward edge of each wing. The footprint is computed once per tree and reused for subsequent chunks.

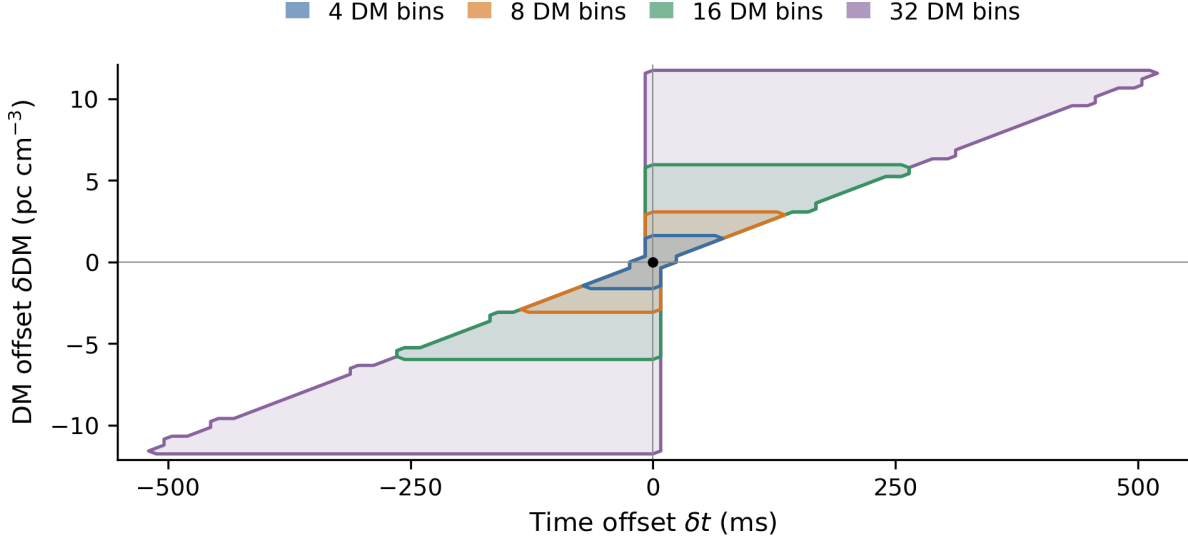


Figure 2. Full-band bow-tie footprints for Tree 0 of the acquisition in Figure 1, with $\text{dm_reach} = 4, 8, 16$ and 32 and $\text{waist_bins} = 1$. Coloured regions enter the local-maximum comparison. The axes express offsets from the central pixel in physical DM and time units. Increasing the DM reach increases both the DM interval and the temporal context required by the filter.

Since a narrowband FRB occupies only part of the observing band, one could instead construct a bank of bow ties and select the footprint associated with the winning subband. I tested this possibility. In simulations of narrowband bursts, both methods recovered about 95% of the bursts, while the bank required substantially more computation. I therefore retained a single full-band footprint per tree.

For some trees, the physical bow tie spans more time columns than a single output chunk. I limit its width by removing the outermost time columns symmetrically until it fits within that tree's native time axis. For example, a 65-column footprint would be reduced to 31 columns for a tree with 32 time bins per chunk: the footprint must have an odd width to remain centred on one pixel. This changes the neighbourhood used for comparison; it does not remove any samples from the input maps. The requested DM-row range is retained, but distant time offsets are no longer compared and some outer DM rows may become empty. The resulting time radius is less than half a native chunk, so the immediately following chunk supplies all the future context needed by the filter. This effective radius also sets the seamless buffer described below.

Peak selection combines two conditions. A pixel must reach the S/N threshold and its value must equal the maximum within its bow-tie neighbourhood. If $\mathbf{S}$ is the input map, $\mathbf{B}$ is the footprint and $\tilde{\mathbf{S}}$ is $\mathbf{S}$ with invalid cells replaced by $-\infty$, the maximum-filtered map is

$$M(d, t) = \max_{(\delta d, \delta t) \in B} \tilde{S}(d + \delta d, t + \delta t).$$

The maximum filter returns S/N values, so comparing its output with the original map gives the local-maximum mask. The implementation also constructs a validity mask $\mathbf{V}$ for every cell: $\mathbf{V}$ is true when the S/N is finite and the argmax token is valid. Token validation checks the profile, frequency multiplet, additional DM and fine-time fields against the plan, including the required

fine-time alignment. These checks are performed on the GPU before peak selection; full decoding into physical parameters is only needed for the surviving candidates. With an S/N threshold τ , the complete selection is

$$C = V \wedge (S \geq \tau) \wedge (S = M).$$

The two conditions remove different unwanted detections. The threshold rejects weak local maxima, while the maximum comparison rejects above-threshold pixels that have a stronger neighbour within the bow tie. A bright burst can therefore produce many threshold crossings but only a few surviving peaks.

Before applying the maximum filter, invalid cells are replaced by -infinite in the competitor map $\hat{\mathbf{S}}$. They cannot be selected and cannot suppress a valid neighbour. All valid cells take part in the comparison, including those below the threshold. If several valid cells have exactly the same maximum S/N, they all survive provided they also reach the threshold.

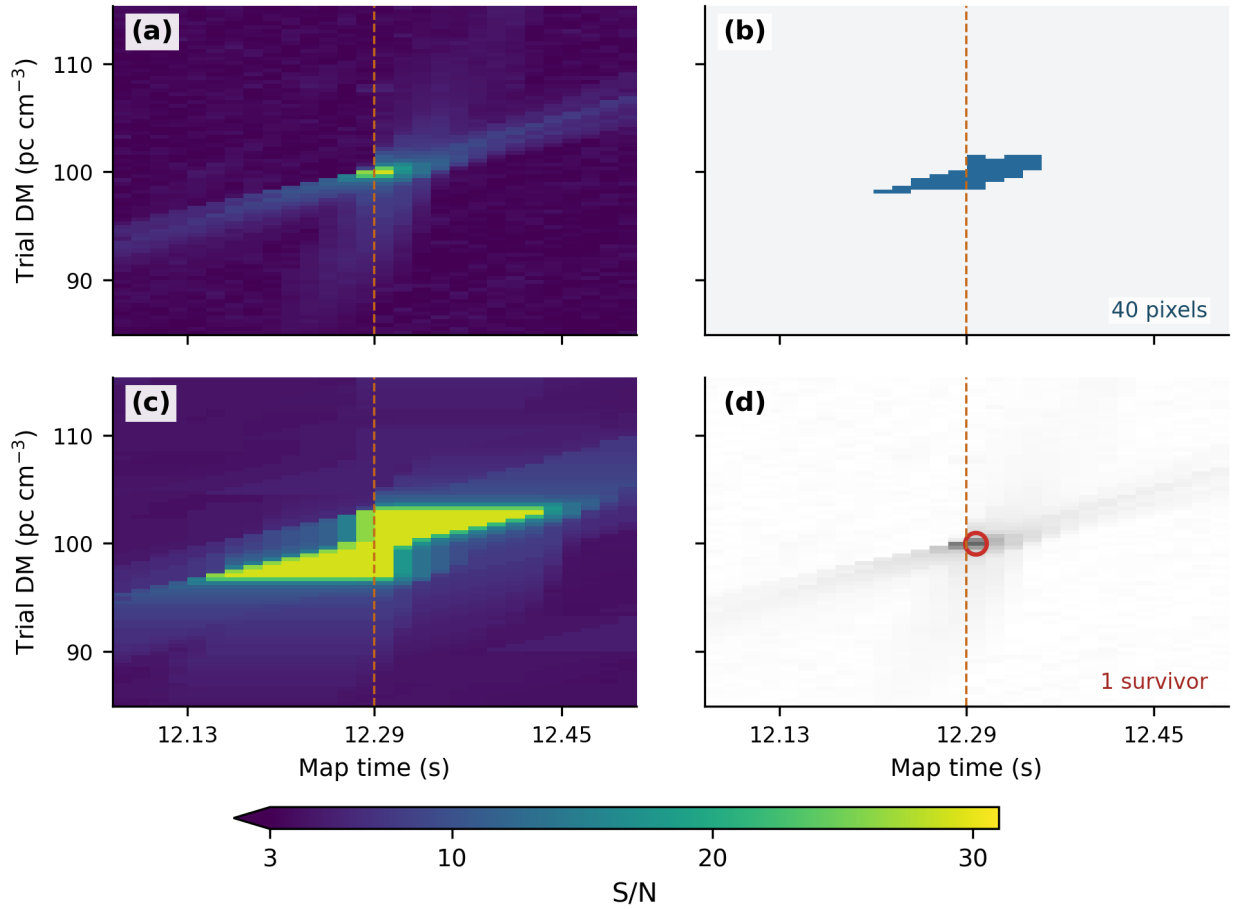


Figure 3. Peak selection around the injected burst in Figure 1. Panels show (a) the original S/N map, (b) the threshold mask, (c) the bow-tie maximum-filtered map and (d) the surviving maximum, marked in red.

The final mask is compacted into candidate arrays containing beam, tree, source chunk, DM/time indices, S/N and argmax token. Missing data before the beginning of an acquisition are handled separately from the validity mask $\mathbf{V}$. A cell with finite S/N and a valid token can still be

selected even if its dedispersion support extends into that missing history. When the producer start is known, the shared processor marks such candidates as `STARTUP_INCOMPLETE` because their S/N and width may be biased. This applies to both offline and online processing. Separate flags record cases where the peakfinding neighbourhood reaches a physical DM or acquisition boundary.

Clustering

After peakfinding, I considered connected-component clustering of the surviving candidates. In this approach, each peak is a vertex in a graph and a pair of compatible peaks is joined by an edge. A connected component contains every peak that can be reached through these links, and its strongest detection can be retained as the representative. Connectivity is therefore defined between candidates in DM and time, rather than by pixel adjacency in the original maps.

The difficulty is that connected components make association transitive. If A is compatible with B and B with C, all three belong to one component even when A and C do not satisfy the pairwise tolerances. A chain of weaker detections can then join two distinct events or bring several peaks from the same tree into one group. Constructing and traversing the candidate graph also adds work and intermediate storage. For this Grouper, I want every member to agree directly with the representative, and I want at most one member from each tree.

I therefore returned to the L1B approach of comparing candidate detections in DM and time and retaining the stronger detection. The implementation here follows that principle, with association rules adapted to pirate's trees. In particular, I restrict comparisons to one beam and primary-tree family, select at most one member per tree, and use the residual-dispersion tolerances defined below. It should therefore be understood as an L1B-style approach, rather than an identical implementation of CHIME L1B.

The candidates are sorted by decreasing S/N and divided by beam and primary-tree family. Within each group, the strongest unassigned candidate becomes the representative. The remaining candidates are compared with it and the strongest compatible detection from each other tree is selected. These selected candidates form one event and are removed from further consideration. The procedure is repeated until all candidates have been assigned. A single candidate can form an event, and an event contains at most one candidate from a given tree.

To define compatibility, let δDM and δt be the signed differences in DM and arrival time, respectively, calculated as candidate minus representative. The required DM condition is

$$|\delta\text{DM}| \leq a_{\text{DM}} \max(\Delta\text{DM}_{\text{cand}}, \Delta\text{DM}_{\text{rep}}).$$

The factor a_{DM} is the configurable `dm_tolerance`. Using the coarser of the two native DM spacings allows for the different discretisation of the trees. The time condition also accounts for

the displacement caused by the DM difference. If s_{lo} and s_{hi} are the residual-delay slopes at the band edges, expressed in full-resolution input samples per unit DM, I require

$$\min(s_{lo}\delta DM, s_{hi}\delta DM) - P \leq \delta t \leq \max(s_{lo}\delta DM, s_{hi}\delta DM) + P,$$

$$P = a_t \max(\Delta t_{cand}, \Delta t_{rep}), \quad s_f = \frac{K_{DM}}{t_{samp}} (f_{ref}^{-2} - f^{-2}).$$

In these expressions δt , the native time spacings and P are all expressed in input samples, and t_{samp} is in seconds. The factor a_t is `time_tolerance`. At the lower reference frequency $s_{lo} = 0$. The sign of δDM matters, since it determines the direction of the predicted time displacement. Both tolerance factors are 1.5 in the example configuration. The decoded pulse width is retained as a measured property and does not set either tolerance.

Every accepted member must satisfy these conditions with respect to the representative. For example, if B is compatible with the representative A and C is compatible only with B, C is not added to A's event. Same-tree peaks and candidates from different beams or primary-tree families remain separate. Thus the clustering does not guarantee one event for every possible extended or multi-component signal; it implements a specific association rule for redundant tree detections.

The event inherits the representative's S/N, DM, arrival time, width, frequency interval and quality flags. The member list preserves the other detections and their individual parameters. I implement the ordered representative selection in a persistent CUDA kernel launched through CuPy. Keeping this loop inside one kernel avoids launching a new sequence of array operations for every representative. The Python implementation section explains how this kernel is combined with the surrounding CuPy operations.

A large population of surviving points can still make clustering expensive. In the worst case, each new representative scans most of the remaining candidates. To prevent an overloaded window from holding up the pipeline indefinitely, a clustering timeout is provided. The example configuration uses `timeout_ms = 400`; setting this value to zero disables it. The timer applies to the persistent clustering kernel, rather than to the complete peakfinding, decoding and output sequence.

By default, the timeout policy discards the events and candidates that entered the timed-out window. It is possible to instead retain only events whose clustering was completed before the interruption. An event already being committed is allowed to finish, so no partially assembled event is returned. Unprocessed candidates are dropped. Since GPU partitions progress independently, the completed events are not necessarily the globally strongest subset.

Seamless handling

The S/N maps arrive as consecutive time chunks, but a burst does not respect these boundaries. Its peak may lie near the end of one chunk while part of the bow-tie neighbourhood lies in the next. Filtering the chunks separately would prevent a pixel close to the boundary from

being compared with all its neighbours. The same burst could then produce one selected peak on each side of the seam, or the selected peak could change once the missing context becomes available.

I handle this by retaining a short time buffer, or halo, for each tree. Let h be the time radius of that tree's effective bow tie. The final h candidate centres are postponed until the next source chunk arrives. Evaluating those centres requires their left-hand context as well as the new right-hand data (see Fig. 4). I therefore retain at least $2h$ columns of S/N and argmax values. The parameter `halo_size` specifies this length as a multiple of h and must be at least 2.

When a new chunk is received, the retained tail is concatenated with its maps and peakfinding is repeated over this extended array. Only centres whose complete temporal neighbourhood is now available and which have not already been emitted are returned. The original source-chunk and local-time indices are retained, so a delayed peak is assigned to the chunk that contained its centre. This avoids treating the overlap as a new set of detections.

Clustering needs an overlap as well, since detections of the same event can be reported on opposite sides of a seam. After receiving chunk $i+1$, I group the candidates from chunk i together with an eligible prefix of $i+1$. For a tree containing N_t coarse time columns, that prefix has length

$$H = \min(\text{halo_size} \times h, N_t - h).$$

The term $N_t - h$ ensures that only peaks already resolved in chunk $i+1$ are included. An event is assigned to chunk i if any of its members belongs to i , even when its strongest member lies in $i+1$. Members used by that event are removed from the next window. Future-only detections are retained for their own chunk. The DM/time tolerances are applied within this temporal domain; increasing them does not extend the halo.

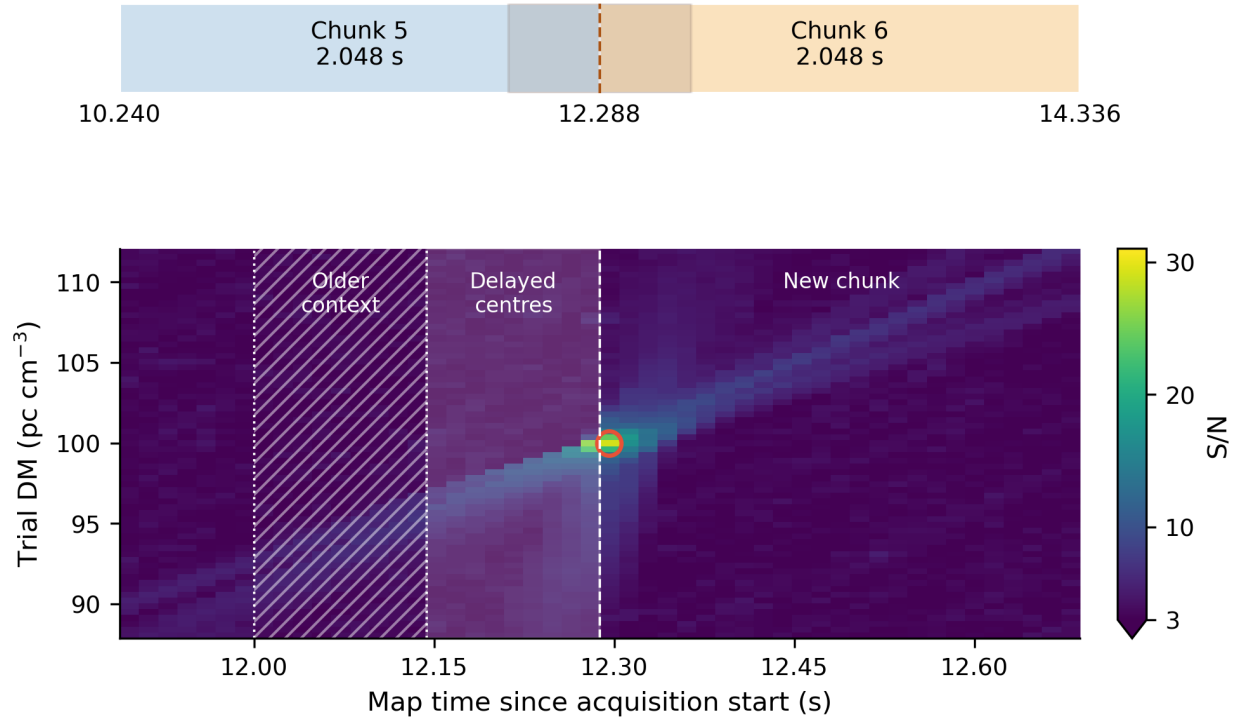


Figure 4. Temporal context around the 12.288 s boundary in the same CHORD Tree 0 maps. The neighbouring chunks each span 2.048 s. The bow-tie time radius is $h = 9$ coarse columns, or 144 ms. With `halo_size = 2`, the retained tail covers 18 columns (288 ms), from 12.000 to 12.288 s. Candidate centres in the final nine columns, from 12.144 to 12.288 s, are delayed until the next chunk supplies their right-hand context. Filtering the two chunks independently selects two peaks from this burst; using the shared temporal context selects one.

This procedure finalises an ordinary chunk after the immediately following chunk has arrived. It therefore keeps both the latency and the retained state bounded. At the physical end of an acquisition, the processor flushes the final delayed centres using the available context and flags the physical right edge. If processing is deliberately stopped after a prefix of an acquisition, that prefix is not flushed as though it were the true end. The current streaming Grouper applies this procedure automatically; `halo_size` controls the retained context rather than a separate on/off seamless option.

Early triggers

Early-trigger trees search the higher-frequency part of the band, where a dispersed burst arrives first. They can report a detection before the pulse reaches the bottom of the complete observing band. The gain in latency is larger at high DM, but the search uses less of the burst's frequency coverage. In the CHORD plan used here, four primary-tree families contain ten search trees in total. The early-trigger bands start at approximately 416, 567 or 750 MHz and extend to 1500 MHz; the full-band trees start at 300 MHz.

The Grouper processes these outputs through the same steps as the full-band trees. Each early-trigger tree has its own native DM/time grid, argmax lookup tables and retained halo. I construct its peakfinding footprint using the same full-band prescription described above and its native spacings. The decoder uses the tree and winning argmax trial to recover the DM, pulse width and frequency interval of the detection.

All decoded arrival times are referred to 300 MHz, including detections made using only the higher-frequency channels. The decoder extrapolates from the winning frequency interval to this common reference and includes the source-chunk offset. An early trigger can therefore have a reported arrival time later than the chunk in which it was detected. That time is the predicted arrival at 300 MHz, while the source-chunk index records where the detection was actually produced. These two quantities serve different purposes and are kept separately.

Early and full-band candidates can be associated when they belong to the same beam and primary-tree family, lie in the current chunk-plus-prefix grouping window, and satisfy the DM/time tolerances. The common arrival-time reference makes that comparison possible. However, the retained window is still limited to adjacent source chunks: the Grouper does not keep an early event pending for the full dispersive delay. It emits that event when its own grouping window is resolved. If the corresponding full-band detection arrives several chunks later, it remains a separate Grouper event; combining those reports would require association over a longer time interval downstream. The same handling, startup flags and timeout policy apply offline and online.

Python implementation

The Python code separates the operations performed on the maps from the orchestration of files, chunks and output buffers. I use CuPy for the arrays and numerical operations that remain on the GPU, while NumPy is used for host-side metadata, file staging and final output. The clustering loop itself is a CUDA kernel launched through CuPy. This arrangement lets the offline and online paths use the same peakfinding, decoding and clustering rules.

Reading and preparing the maps

`OfflineMapReader.py` handles the offline input. It discovers the ASDF maps, checks beam/chunk consistency and reconstructs the producer plan. Its `load_beam_chunk()` method loads a compatible group of beams for one chunk. The file arrays are copied into owned NumPy staging arrays because ASDF data may be loaded lazily and tied to an open file handle. The staged arrays are then uploaded to CuPy, one S/N and argmax array per tree. The processing batch therefore has predictable ownership without keeping the entire acquisition in host or device memory.

`GrouperConfig.py` reads the peakfinding, grouping and execution parameters from YAML for both offline and online processing. `GrouperPipeline.py` builds the tree geometries and decoder once through `GrouperSetup`, then creates a `StreamingGrouper` for each beam batch. The streaming processor retains map tails and pending candidates, and emits each completed association window as a `ClusteringWindow`. Keeping this state in one common component

prevents the offline and online implementations from developing different seam or timeout behaviour.

Finding, decoding and clustering candidates

`BowtiePeakfinding.py` contains the bow-tie construction and the `StreamingPeakExtractor`. Geometry construction uses the plan on the CPU to compute small, fixed footprints and lookup tables, which are uploaded once. During processing, finite-value checks, token validation, `maximum_filter`, Boolean selection, candidate compaction and ordering use CuPy arrays. The beam axis is kept separate in the footprint, so one beam cannot suppress a peak in another.

`GpuArgmaxDecoder.py` translates the selected coarse coordinates and argmax tokens into physical event parameters. It builds small plan-dependent lookup tables during setup, then decodes the candidate columns on the GPU. This avoids decoding every map pixel or copying the full candidate list to the CPU simply to interpret its tokens. The source chunk remains part of the calculation, which is necessary for an absolute arrival time when peaks are emitted one chunk later.

`Clustering.py` implements the representative-seeded association described above through `cluster_candidates()`. Candidate sorting and partition construction use CuPy arrays. CuPy `RawModule` compiles and launches the custom CUDA kernel, which keeps the ordered selection loop on the device while threads within each beam/family partition compare candidates in parallel. Expressing each iteration as separate CuPy array operations would require repeated launches and coordination between Python and the GPU as representatives are selected. The persistent kernel keeps those decisions and the assignment state together. CuPy operations then restore the deterministic event order and construct the event and membership tables. Both offline and online adapters call this same implementation.

Running offline and online

`run_offline_grouper.py` is the offline entry point. It loads the configuration, iterates over consecutive chunks in bounded beam batches, passes them through the common processor, and optionally writes an ASDF trigger catalog. The main callable is `run_offline_grouper(acqdir, config_file, ...)`, with options for the CUDA device, a leading `max_chunks` subset and an output catalog. A subset is useful when inspecting a long acquisition, but its unresolved final window is not treated as a physical acquisition end.

`OnlineGrouper.py` connects the common processor to the live producer. Within `FrbGrouper.get_output()`, `out_max` and `out_argmax` are CuPy views of the shared GPU ring buffers. They are consumed directly, without staging the full maps through NumPy. The live beam batches follow the producer's batches. Before the output context is released, the current CuPy stream is synchronized so that the producer cannot overwrite data still being read by the consumer.

The online path also requires Grouper settings. They are read from the grouper section of the live YAML recipe and passed through the same configuration parser. The dedispersion geometry comes from the producer plan, so it does not need to be restated in a separate

Grouper plan. The offline `beam_batch_size` setting controls file-processing batches; online batches follow the live producer.

Some GPU copies are nevertheless necessary. The retained halo must own its storage after the producer recycles the input buffer, and the selected candidate arrays must likewise survive that release. The short map tails are therefore copied and the selected candidate values gathered on the GPU. The extended work map and the maximum-filtered map also require temporary device storage. These are device operations; they are distinct from transferring the maps to the CPU.

`TriggerCatalog.py` converts completed event, candidate and membership columns to NumPy for output. The online adapter currently uses the same catalog conversion before assembling the representative event messages, so it also materialises member information on the host even though the transmitted event message contains the representative parameters. A small per-beam map maximum is transferred separately for monitoring. These host conversions, together with scalar validity/count/status checks, are real synchronization points and are included where specified in the wall-time measurements below.

The following settings illustrate the controls normally adjusted for a run. The two tolerance values are dimensionless factors of the native search spacing, and `halo_size` is a radius multiplier, rather than a number of input samples.

Parameter	Example value	Role
<code>snr_threshold</code>	10	Minimum S/N of a selected peak.
<code>dm_reach</code> ; <code>waist_bins</code>	8; 1	Local bow-tie neighbourhood.
<code>dm_tolerance</code> ; <code>time_tolerance</code>	1.5; 1.5	Cross-tree DM/time agreement.
<code>halo_size</code>	2	Retained context in units of time radius.
<code>beam_batch_size</code>	1	Maximum compatible offline beam batch.
<code>timeout_ms</code> ; <code>timeout_policy</code>	400; discard	Clustering deadline and incomplete-output policy.

Performance

I consider separately the cost of filtering the full maps, the association of an already decoded candidate list, and the wall time of the shared-output consumer. All GPU measurements below use an NVIDIA A40. The benchmark plan contains ten trees covering 300–1500 MHz. For the peakfinding scan, one acquisition chunk contains 2048 samples at 1 ms, giving a processing budget of 2.048 s for all 60 beams. The reported load is the measured processing time divided by this budget.

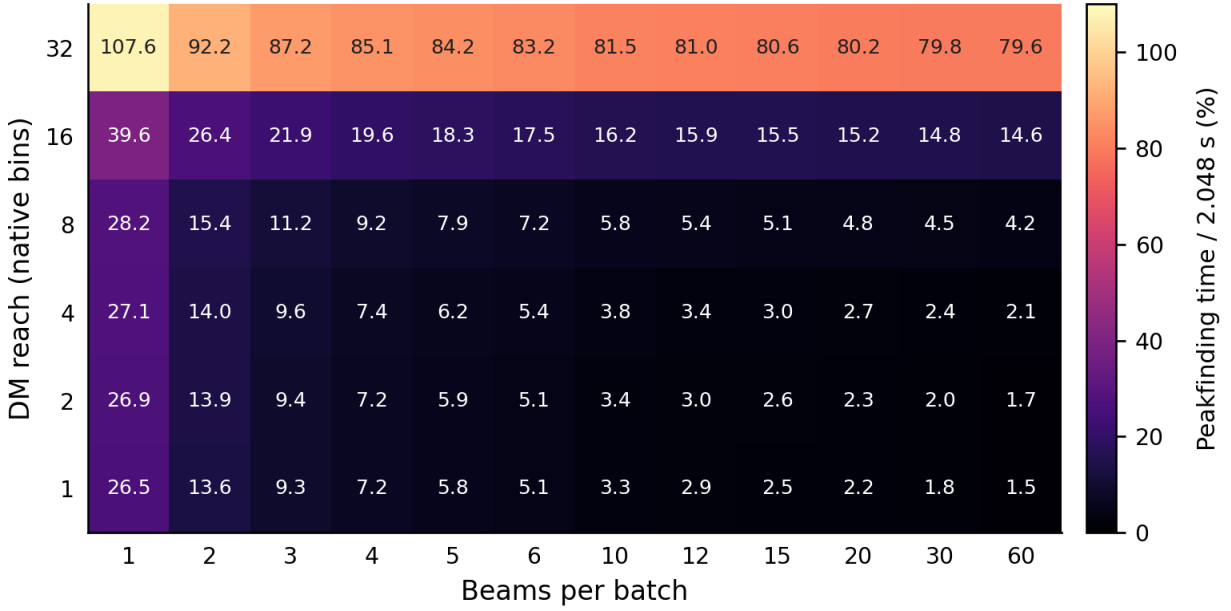


Figure 5. Peakfinding time as a percentage of the 2.048 s chunk duration, for 60 beams and all ten trees. The horizontal axis gives the number of beams processed together; the vertical axis gives the native DM reach. Each cell uses the median of ten iterations after one warm-up. The input consists of independent Gaussian S/N maps with no values reaching the threshold of 10; waist_bins = 1. This figure measures the map-processing cost in the absence of selected candidates. Timing is synchronized and includes streaming bookkeeping and candidate concatenation; setup, input upload, decoding, clustering and output are excluded.

Figure 5 shows that batching reduces the cost substantially. At dm_reach = 8, processing one beam at a time uses 28.2% of the chunk budget, compared with 15.4% for batches of two and 4.2% for all 60 beams together. Larger footprints remain expensive: dm_reach = 32 uses about 80% even with large batches. These tests produced no candidates, so they do not measure the additional cost of a crowded candidate list.

To measure Clustering with a non-empty input, I timed the GPU implementation on synthetic decoded candidate tables. I varied the number of candidates from 300 to 4800 and distributed them over either 60 or two beams, using the same ten-tree geometry and DM/time tolerance factors of 1.5. Half of the rows were generated in compatible cross-tree pairs; the others were independent points within the native DM and time ranges.

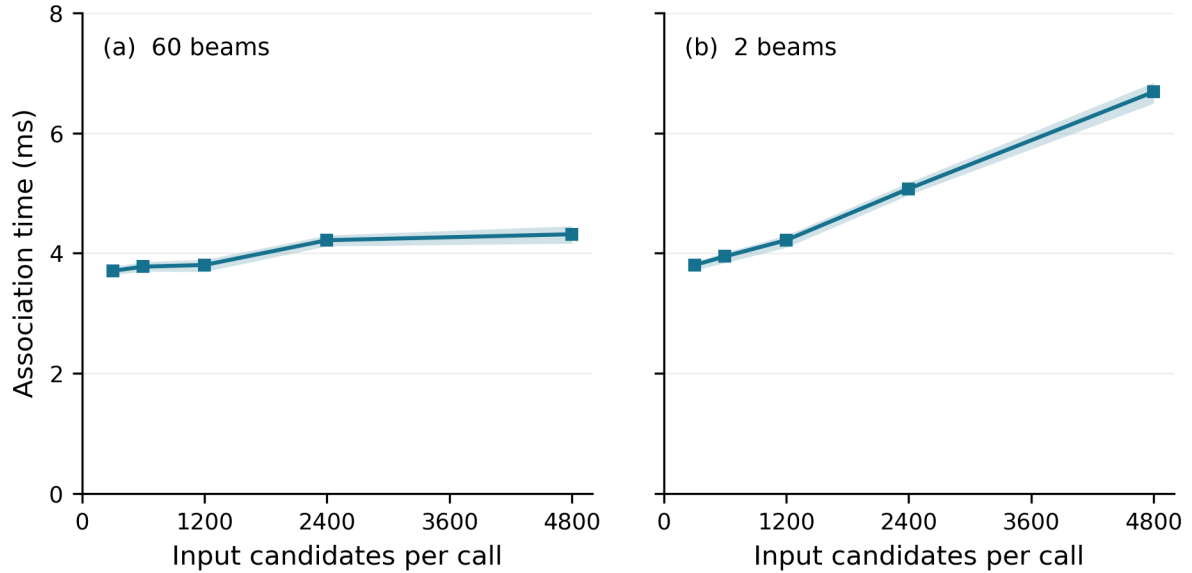


Figure 6. Synchronized wall time for one association call, with the input candidates distributed over 60 beams (a) or two beams (b).

At 300 candidates, the GPU takes approximately 3.7–3.8 ms. At 4800 candidates distributed over 60 beams, the time is 4.3 ms. Concentrating the same number of candidates into two beams increases the time to 6.7 ms. The concentration matters because the work is partitioned by beam and primary-tree family: the largest partition contains about 46 candidates in the first case and 889 in the second. The GPU still has to advance through the representative-selection loop within each partition. In Figure 6, the candidate count is the total in one call. Panel (b) therefore times only two beams. Processing 60 beams in batches of two requires 30 such calls per grouping window, in addition to peakfinding, decoding and output. Those full 60-beam association schedules are measured in Figure 7.

I extended the GPU benchmark to exactly 100, 300 and 1000 surviving peaks in each of the 60 beams, giving 6000, 18,000 and 60,000 candidates in total. For each population I timed both one call containing all beams and a sequence of 30 calls containing two beams each. The latter measures the association cost for the beam batching used in the consumer test below. The synthetic candidate model and tolerances are the same as in Figure 6.

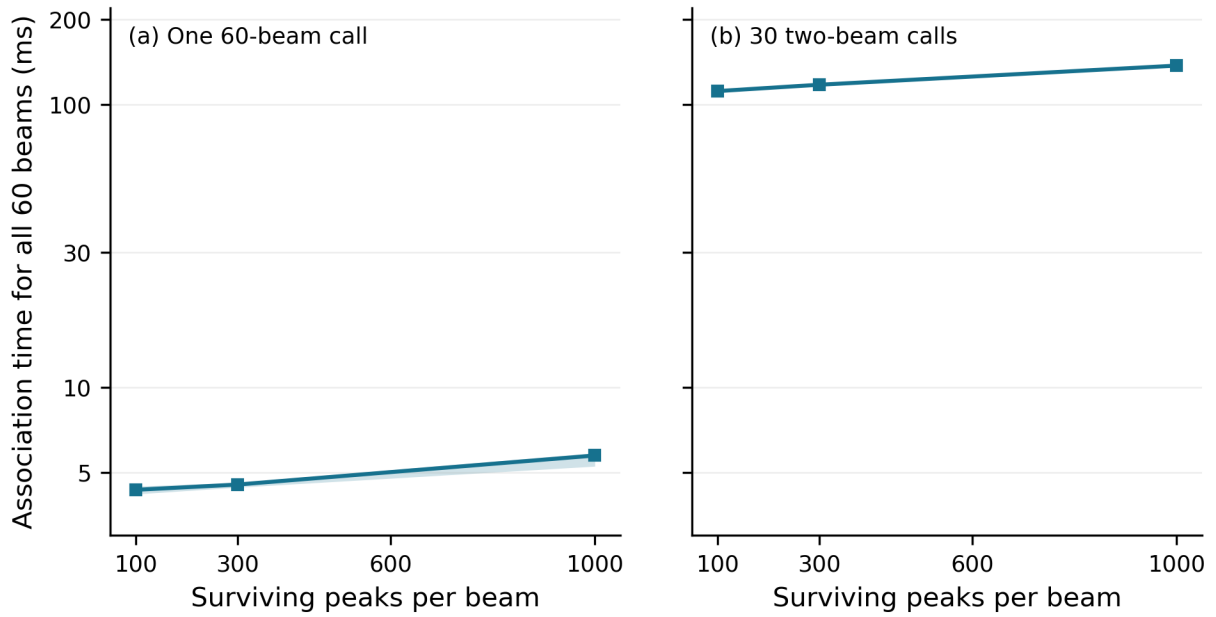


Figure 7. Association wall time for all 60 beams, with the stated number of peaks in every beam. Panel (a) processes all beams in one call; panel (b) sums the work of 30 sequential two-beam calls.

At 1000 peaks per beam, the GPU processes the full 60,000-candidate table in 5.7 ms in one call. With two-beam batches, the total GPU time is 137.5 ms. The batched GPU association alone uses about 6.7% of a 2.048 s chunk. No call timed out.

The difference between the two GPU schedules is substantial even though they contain the same candidates. One call exposes all beam/family partitions together and pays the call overhead once. Small batches expose fewer partitions at a time and repeat sorting, partition construction and synchronization.